Ek 2 Yerel Kubernetes Uygulama Kılavuzu

Bulut Bilişim • Hafta 6 • Windows ve PowerShell

Bu kılavuz, Hafta 5 Ders Notları Kayıt Uygulaması'nın web ve PostgreSQL bileşenlerini yerel Kubernetes kümesinde çalıştırır. Dağıtımın ardından kayıt işlemleri, kalıcılık, sağlık kontrolleri, Pod yenileme, elle ölçekleme, sürüm güncelleme ve geri dönüş deneylerini tamamlayacaksınız. Son bölümde ders notunun 7.1 ve 7.3 bölümlerindeki görevlerin karşılıkları ve teslim içeriği bulunur.

Ek 1'deki yerel ortam hazırlığının tamamlandığı varsayılır. Yerel uygulama zorunludur; bulut görevleri Ek 3 kapsamındadır. Windows'a ayrıca Python veya PostgreSQL kurulmaz. Komutlar normal PowerShell'de çalıştırılır. Dosya içerikleri belirtilen dosyalara UTF-8 olarak kaydedilir; YAML girintilerinde sekme yerine boşluk kullanılır.

1 Çalışma düzeni

Hafta 5'in çalışma klasörünü koruyun. Hafta 6 için onun bir kopyasını kullanın. İmajların 1.1 ve 1.2 etiketleri bu kılavuzda uygulamanın gerçekten gösterdiği sürümlerle eşleştirilir. Eski bir etiketi farklı içerikle yeniden kullanmayın; önceki imaj geri dönüş için erişilebilir kalmalıdır.

Öğe	Bu çalışmada kullanılan değer
Namespace	hafta6
Web ve veritabanı	web ve db Deployment; her biri başlangıçta 1 kopya
Service ve port	web:8000; db:5432; ikisi de ClusterIP
Tarayıcı	http://localhost:8080; geçici port yönlendirme
Yapılandırma ve gizli bilgi	app-config; app-secrets
Kalıcı depolama	db-data PVC; varsayılan StorageClass üzerinden PV
İmajlar	dersnotlari-web:1.1 ve dersnotlari-web:1.2; postgres:16

Kubernetes çalışma dosyalarını k8s klasöründe tutun. Gizli bilgiler secrets klasöründe kalır ve rapora eklenmez. Her deneyin önceki ve sonraki durumunu kaydedin; sonraki deneye geçmeden uygulamanın tekrar hazır olduğunu kontrol edin.

2 Kaynakları ve imajı hazırlama

Ek 1'de oluşturduğunuz Hafta6 klasörünü kullanın. Aşağıdaki kaynak yolunu Hafta 5 çalışmanızın gerçek konumuna göre değiştirin. Hedefte mevcut çalışma varsa kopyalama işlemini tekrar çalıştırmayın.

```
$week5Path = Join-Path $env:USERPROFILE "bulut-dersi\Hafta-5-Uygulama"
$week6Path = Join-Path $env:USERPROFILE "BulutDersi\Hafta6"
New-Item -ItemType Directory -Force -Path $week6Path | Out-Null
Copy-Item -Path "$week5Path\*" -Destination $week6Path -Recurse
Copy-Item -LiteralPath "$week5Path\.dockerignore" -Destination $week6Path
Set-Location $week6Path
Get-ChildItem -Force
New-Item -ItemType Directory -Force -Path .\k8s | Out-Null
kubectl config current-context
kubectl get nodes
kubectl get storageclass
```

Beklenen sonuç: app.py, requirements.txt, Dockerfile, .dockerignore, templates, static ve secrets bulunur. Bağlam docker-desktop veya Ek 1'deki Minikube hafta6 profilidir; düğüm Ready durumundadır. Varsayılan depolama sınıfı görünmelidir. Varsayılan yoksa PVC dosyasına Ek 1'de doğruladığınız sınıfı storageClassName alanıyla ekleyin; rastgele bir sınıf adı kullanmayın.

2.1 İlk sürümü oluşturma

```
notepad .\app.py
docker version
```

app.py içinde APP_VERSION değerini "1.1" yapın ve kaydedin. Dockerfile ile .dockerignore Hafta 5 Ek 2'deki içerikleri korur. .dockerignore içine aşağıdaki satırı ekleyin; secrets/ ve .env dışlama satırları da bulunmalıdır:

```
k8s/
```

Sonra aşağıdaki komutları çalıştırın:

```
docker build -t dersnotlari-web:1.1 .
docker image ls dersnotlari-web
docker run --rm dersnotlari-web:1.1 python -c "import flask, psycopg; print('Hazir')"
```

Beklenen sonuç: 1.1 imajı oluşur, kütüphane kontrolü Hazir yazdırır. Bu kontrol veritabanı bağlantısını sınamaz. Hafta 5 dosyalarınız yoksa kaynak paketini Hafta 5 Ek 2'ye göre hazırlayıp Dockerfile ve .dockerignore dosyalarını oluşturun; bu kılavuz hazır bir Hafta 5 çalışmasını kullanır.

3 İmajı yerel kümeye erişilebilir kılma

İmajın Docker listesinde bulunması, kümenin onu kullanabileceğini tek başına göstermez. Kullandığınız ortam için yalnız uygun yolu izleyin [1], [2].

İmaj erişimi için yalnız kullandığınız ortama ait alt bölümü uygulayın. Ardından bütün öğrenciler Bölüm 4'teki ortak hazırlıkla devam eder.

3.1 Docker Desktop kubeadm

Ek 1'deki ana yol kubeadm ise yerel Docker imajını kullanarak Bölüm 4'e geçin. Web tanımında imagePullPolicy: Never, yanlışlıkla bir dış depodan aynı etiketi indirmeyi engeller. Dağıtımda ErrImageNeverPull görülürse kullanılan imaj deposu ve küme türünü inceleyin; imajı erişilebilir kılmadan devam etmeyin.

3.2 Minikube alternatifi

```
minikube image load -p hafta6 dersnotlari-web:1.1
minikube image ls -p hafta6
```

Beklenen sonuç: küme imaj listesinde dersnotlari-web:1.1 görünür. 1.2 sürümü hazırlandığında aynı yükleme işlemi yeni etiket için de yapılacaktır [2].

3.3 Mevcut Docker Desktop kind kümesi

kind kullananlar için yerel imajın düğümlere ayrıca yüklenmesi gerekebilir. kind CLI kuruluysa aşağıdaki komutlarla küme adını kontrol edin. Yoksa winget install -e --id Kubernetes.kind ile kurup PowerShell'i yeniden açın. Yeni terminalde çalışma klasörüne tekrar geçin.

```
kind get clusters
kind load docker-image dersnotlari-web:1.1 --name desktop
```

İkinci komuttaki desktop yalnız listede gerçekten görünen küme adınızsa kullanılır; farklıysa onu yazın. kind get clusters mevcut kümeyi göstermiyorsa bu yolu zorlamayın. Ek 1'deki Minikube alternatifiyle hazırlanmış bir yerel küme kullanın; diğer kümeyi veya imaj deposunu sırf bu işlem için sıfırlamayın. kind yolu için imaj tüm ilgili düğümlere yüklenir [3].

4 Yapılandırma ve gizli bilgileri tanımlama

Bütün çalışma ortamları için ortak hazırlık

ConfigMap ve Secret nesnelerini oluşturmadan önce, doğru yerel kümeye bağlı olduğunuzu kontrol edin ve uygulamanın kullanacağı hafta6 ad alanını oluşturun. Bu adım, Docker Desktop ve Minikube kullanan bütün öğrenciler için gereklidir.

```
kubectl config current-context
kubectl get namespaces
kubectl create namespace hafta6
kubectl get namespace hafta6
```

Beklenen sonuç: hafta6 ad alanı listelenir. Oluşturma komutu AlreadyExists mesajı veriyorsa, doğru kümede ve kendi çalışma ad alanınızda olduğunuzu doğrulayarak devam edin. Ardından 4.1'deki ConfigMap ve 4.2'deki Secret adımlarını uygulayın.

4.1 ConfigMap dosyası

k8s\01-config.yaml dosyasını oluşturup aşağıdaki içeriği kaydedin. ConfigMap (yapılandırma haritası), gizli olmayan uygulama ayarlarını tutar [4].

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: hafta6
data:
  DB_HOST: db
  DB_PORT: "5432"
  DB_NAME: dersnotlari
  DB_USER: dersuser
  COOKIE_SECURE: "false"
```

Sonra aşağıdaki komutları uygulayın:

```
kubectl apply -f .\k8s\01-config.yaml
kubectl get configmap app-config -n hafta6
```

4.2 Secret oluşturma

Hafta 5'teki db_password.txt ve app_secret.txt dosyalarını kullanın. Bunların içeriğini ekrana yazdırmayın. Yeni bir Kubernetes depolama alanı oluşturulacağı için Hafta 5 Docker volume'undaki kayıtlar aktarılmaz. Küme Secret nesnesini aşağıdaki dosyalardan üretir [5].

```
Get-ChildItem .\secrets | Select-Object Name, Length
kubectl create secret generic app-secrets -n hafta6 --from-file=db_password=./secrets/db_password.txt --from-file=app_secret=./secrets/app_secret.txt
kubectl get secret app-secrets -n hafta6
```

Beklenen sonuç: ConfigMap oluşur; Secret için DATA=2 görünür. Secret zaten varsa dosyaları değiştirmeden mevcut çalışmayı inceleyin. Yeni parola üretmek, mevcut PostgreSQL veritabanı parolasını kendiliğinden değiştirmez. Secret'in base64 gösterimi şifreleme değildir; -o yaml çıktısını rapora almayın.

Web iki gizli dosyaya erişir; PostgreSQL yalnız veritabanı parolasına erişir. APP_SECRET uygulamanın form oturumlarını imzalar. Dosyalar web konteynerinde /run/secrets yoluna bağlanır; uygulama bu yolları DB_PASSWORD_FILE ve APP_SECRET_FILE değişkenleriyle okur.

5 Kalıcı depolama ve veritabanı

5.1 PVC dosyası

k8s\02-pvc.yaml dosyasını aşağıdaki içerikle oluşturun. PVC — PersistentVolumeClaim (kalıcı birim talebi), varsayılan StorageClass üzerinden bir PV — PersistentVolume (kalıcı birim) ayrılmasını ister. PV'yi ayrıca elle oluşturmayız [6].

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
  namespace: hafta6
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Sonra aşağıdaki komutları uygulayın:

```
kubectl apply -f .\k8s\02-pvc.yaml
kubectl get pvc -n hafta6
kubectl get pv
```

Beklenen sonuç: PVC Bound olur; WaitForFirstConsumer kullanan sınıflarda tüketici Pod yerleştirilene kadar Pending kalması normal olabilir. Sonraki dağıtımdan sonra tekrar kontrol edin. ReadWriteOnce tek düğümde yazma erişimini ifade eder; tek Pod garantisi olarak yorumlanmaz.

5.2 Veritabanı tanımını hazırlama

Sonraki sayfadaki içeriği k8s\03-db.yaml dosyasına kaydedin. PostgreSQL bu eğitim çalışmasında tek kopyalı Deployment ve Recreate stratejisiyle çalışır. Web gibi çoğaltmayın; bu yapı veritabanı çoğaltma veya yüksek erişilebilirlik düzeni değildir.

Veri dizini /var/lib/postgresql/data altına bağlanır. PGDATA alt dizini, boş olmayan bir bağlama kökündeki başlangıç sorunlarını azaltır. POSTGRES_PASSWORD_FILE, yalnız ilk veri alanı hazırlanırken parolayı belirler. Dosya sonradan değişirse mevcut veritabanı parolası otomatik değişmez.

5 Veritabanı dosyasının içeriği

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db
  namespace: hafta6
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      automountServiceAccountToken: false
      containers:
        - name: db
          image: postgres:16
          env:
            - {name: POSTGRES_DB, value: dersnotlari}
            - {name: POSTGRES_USER, value: dersuser}
            - name: POSTGRES_PASSWORD_FILE
              value: /run/secrets/db_password
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          ports:
            - containerPort: 5432
          readinessProbe:
            exec:
              command: [pg_isready, -U, dersuser, -d, dersnotlari]
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests: {cpu: 100m, memory: 128Mi}
            limits: {cpu: 500m, memory: 512Mi}
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data}
            - {name: secret, mountPath: /run/secrets, readOnly: true}
      volumes:
        - name: data
          persistentVolumeClaim: {claimName: db-data}
        - name: secret
          secret:
            secretName: app-secrets
            items:
              - {key: db_password, path: db_password}
```

6 Service nesnelerini ve web dağıtımını hazırlama

6.1 Service dosyası

k8s\04-services.yaml dosyasına iki nesneyi birlikte kaydedin. Üç tire, aynı dosyada ayrı YAML belgelerini ayırır. Selector (seçici), Pod etiketleriyle eşleşir; targetPort Pod içindeki porttur [7].

```
apiVersion: v1
kind: Service
metadata:
  name: db
  namespace: hafta6
spec:
  type: ClusterIP
  selector:
    app: db
  ports:
    - {port: 5432, targetPort: 5432}
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: hafta6
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - {port: 8000, targetPort: 8000}
```

6.2 Web dosyasını hazırlama

Sonraki iki sayfadaki parçaları sırayla aynı k8s\05-web.yaml dosyasına kaydedin. İlk parça tek başına tamamlanmış tanım değildir. Bir sonraki sayfanın açıklama metinlerini dosyaya eklemeyin.

Web için readiness probe (hazır olma kontrolü) /health/ready adresiyle veritabanı bağımlılığını sınar. Liveness probe (canlılık kontrolü), /health/live adresini sorgular. Ayrıca yalnız kontrollü ders deneyi için /tmp/live-fail dosyasının bulunmamasını ister. Bu dosya normalde yoktur; Bölüm 10'da oluşturulunca canlılık kontrolünü başarısız yapar [8].

6 Web dosyasının ilk parçası

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: hafta6
spec:
  replicas: 1
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      automountServiceAccountToken: false
      containers:
        - name: web
          image: dersnotlari-web:1.1
          imagePullPolicy: Never
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_PASSWORD_FILE
              value: /run/secrets/db_password
            - name: APP_SECRET_FILE
              value: /run/secrets/app_secret
          startupProbe:
            httpGet:
              path: /health/live
              port: 8000
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet:
              path: /health/ready
              port: 8000
            periodSeconds: 5
            timeoutSeconds: 10
            failureThreshold: 2
```

6 Web dosyasının devamı

Aşağıdaki ilk satır, readinessProbe ile aynı girinti düzeyindedir. volumeMounts konteynere; volumes Pod spec bölümüne aittir.

```
    livenessProbe:
      exec:
        command:
          - python
          - -c
          - >-
            import pathlib, urllib.request;
            assert not pathlib.Path('/tmp/live-fail').exists();
            urllib.request.urlopen(
            'http://127.0.0.1:8000/health/live', timeout=2).read()
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 3
    resources:
      requests: {cpu: 100m, memory: 128Mi}
      limits: {cpu: 500m, memory: 256Mi}
    volumeMounts:
      - name: secret
        mountPath: /run/secrets
        readOnly: true
  volumes:
    - name: secret
      secret:
        secretName: app-secrets
```

6.3 Tanımların anlamı

startupProbe (başlangıç kontrolü), web sürecinin başlamasını bekler; veritabanının hazır olmasını canlılık şartı yapmaz. Başlangıç kontrolü başarılı oluncaya kadar diğer iki kontrol çalışmaz. Veritabanı erişilemiyorsa web canlı kalabilir, ancak hazır olma kontrolü başarısız olur.

100m CPU, bir işlemci çekirdeğinin onda biri kadar taleptir; 128Mi, 128 mebibayt bellek talebidir. İki web Pod'unun toplam talebi 200m CPU ve 256Mi bellektir; veritabanı ve sistem bileşenleri bu hesaba eklenir. Bunlar eğitim örneği değerlerdir; gerçek yük ölçümüyle belirlenmiş üretim boyutlandırması değildir [9].

ConfigMap değerleri ortam değişkeni olarak başlangıçta alınır. ConfigMap'i değiştirmek çalışan sürecin ortamını kendiliğinden güncellemez. Bu çalışmada gizli bilgi dosyaları değiştirilmez; bir parola değişikliği ayrıca veritabanı ve uygulama uyumu gerektirir.

7 Dağıtım ve ilk doğrulama

7.1 Tanımları kontrol edip uygulama

```
kubectl apply --dry-run=client -f .\k8s
kubectl apply -f .\k8s\03-db.yaml
kubectl apply -f .\k8s\04-services.yaml
kubectl apply -f .\k8s\05-web.yaml
kubectl rollout status deployment/db -n hafta6 --timeout=180s
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get deployments,pods,services,pvc -n hafta6
kubectl get endpointslices -n hafta6
```

Beklenen sonuç: db ve web Deployment için READY=1/1; iki Pod için 1/1 Running; db-data Bound; web ve db ClusterIP Service'leri bulunur. İlk indirmeler uzun sürerse zaman aşımı tek başına başarısız dağıtım anlamına gelmez; olayları inceleyin. Dry-run dosya ve istemci kontrolüdür, çalışma başarısının kanıtı değildir.

7.2 Tarayıcı erişimi

Ayrı bir PowerShell penceresinde aynı küme bağlamını doğrulayın ve aşağıdaki komutu çalışır bırakın [10]:

```
kubectl port-forward -n hafta6 service/web 8080:8000 --address 127.0.0.1
```

Tarayıcıda http://localhost:8080 adresini açın. İki test kaydı oluşturun; numara, başlık ve açıklamalarını not edin. Örneğin Kalıcılık A ve Kalıcılık B başlıkları kullanın. Sayfayı yenileyin; iki kayıt listelenmeli ve sürüm 1.1 görünmelidir.

```
Invoke-RestMethod -Uri 'http://localhost:8080/health/ready'
```

Beklenen sonuç: status=ready, version=1.1. Port yönlendirme penceresi kapanırsa erişim kesilir. Pod değiştirildiğinde oturum sonlanabilir; Ctrl+C ile kapatıp aynı komutla yeniden açın. 8080 doluysa yalnız ilk portu 8081 yapın ve tarayıcı adresini buna göre değiştirin.

Kanıt: nesne durumu, iki kayıt ve sürüm ekranı. Tarayıcı erişimi tek başına Service'in normal trafik yolunu doğrulamaz; sonraki adımda küme içi istemci kullanılır.

8 Service üzerinden küme içi erişimi doğrulama

Normal trafik yolunu ayrıca sınamak için geçici bir kontrol Pod'u oluşturun. Web ile aynı imajı kullanır, ancak uygulamayı başlatmak yerine bekler; gizli bilgi bağlanmaz. k8s\06-check.yaml dosyasını kaydedin.

```
apiVersion: v1
kind: Pod
metadata:
  name: check
  namespace: hafta6
spec:
  restartPolicy: Never
  containers:
    - name: check
      image: dersnotlari-web:1.1
      imagePullPolicy: Never
      command: [python, -c, "import time; time.sleep(86400)"]
      resources:
        requests: {cpu: 10m, memory: 32Mi}
        limits: {cpu: 100m, memory: 64Mi}
```

```
kubectl apply -f .\k8s\06-check.yaml
kubectl wait -n hafta6 --for=condition=Ready pod/check --timeout=120s
kubectl exec -n hafta6 check -- python -c "import urllib.request; print(urllib.request.urlopen('http://web:8000/health/ready', timeout=10).read().decode())"
kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
```

Beklenen sonuç: küme içindeki check Pod'u web Service adı üzerinden ready ve 1.1 yanıtını alır. EndpointSlice içinde web Pod adresi ve ready: true görülür. Bu çıktı gizli bilgi içermez; trafik hedeflerini incelemek için kullanılabilir [7].

check Pod'u bir gün sonra kendiliğinden tamamlanır; deney süresi uzarsa silip aynı dosyayla yeniden oluşturun. Tamamlanmış veya başarısız check Pod'una exec çalışmaz. Bu Pod deneyler boyunca kalır ve çalışmanın sonunda kaldırılır.

9 Veri kalıcılığı ve web Pod yenileme

9.1 PostgreSQL Pod'unu yeniden oluşturma

```
kubectl get pods -n hafta6 -l app=db -o wide
kubectl get pvc db-data -n hafta6 -o wide
kubectl get pods -n hafta6 -l app=db -o jsonpath='{.items[0].metadata.uid}'
kubectl delete pod -n hafta6 -l app=db
kubectl rollout status deployment/db -n hafta6 --timeout=180s
kubectl wait -n hafta6 --for=condition=Ready pod -l app=web --timeout=180s
kubectl get pods -n hafta6 -l app=db -o wide
kubectl get pods -n hafta6 -l app=db -o jsonpath='{.items[0].metadata.uid}'
kubectl get pvc db-data -n hafta6 -o wide
```

Beklenen sonuç: yeni db Pod'u farklı ad ve UID — Unique Identifier (benzersiz kimlik) taşır; PVC ve bağlı PV aynı kalır. Tarayıcıyı yenileyin: Kalıcılık A ve B kayıtları aynı numaralarla görünür. Pod yeniden oluşturulurken geçici erişim hatası olabilir; toparlanma sonrasında doğrulayın. PVC'yi bu deneyde silmeyin.

Kanıt: eski ve yeni Pod kimlikleri, değişmeyen PVC/PV ve aynı kayıtların ekranları. Kalıcılık yedekleme değildir; küme veya depolama kaybı ayrıca ele alınır.

9.2 Web Pod'unu silme

```
kubectl get pods -n hafta6 -l app=web -o wide
kubectl get pods -n hafta6 -l app=web -o jsonpath='{.items[0].metadata.uid}'
kubectl delete pod -n hafta6 -l app=web
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get pods -n hafta6 -l app=web -o wide
kubectl get pods -n hafta6 -l app=web -o jsonpath='{.items[0].metadata.uid}'
```

Beklenen sonuç: Deployment'ın yönettiği ReplicaSet eksik kopyayı karşılar; yeni web Pod'u oluşur ve hazır olur. Eski port yönlendirme bağlantısını yeniden açın; kayıtları kontrol edin. Web Pod'u içindeki yazılabilir katman kayıtların saklandığı yer değildir.

10 Sağlık kontrollerinin farklı sonuçları

10.1 Veritabanı kesintisi ve hazır olma

```
kubectl get pods -n hafta6 -l app=web
kubectl scale deployment/db -n hafta6 --replicas=0
kubectl get pods -n hafta6
kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
```

Web hazır olma kontrolünün iki kez başarısız olmasını bekleyin; bağlantı zaman aşımı nedeniyle bu birkaç kontrol aralığı sürebilir. Komutları yeniden çalıştırın. Beklenen sonuç: db Pod'u yok; web Running olsa da READY=0/1; web EndpointSlice hedefinde ready: false. Web yeniden başlatma sayısı, başka bir arıza yoksa artmaz. Bu kesinti canlılık şartını bozmaz.

```
kubectl exec -n hafta6 deployment/web -- python -c "import urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8000/health/live').read().decode())"
kubectl exec -n hafta6 check -- python -c "import urllib.request; print(urllib.request.urlopen('http://web:8000/health/ready',
timeout=10).read().decode())"
```

İlk komut status=ok döndürür. İkinci komut hazır hedef olmadığı için bağlantı hatası/zaman aşımı verebilir; bu deneyde beklenir. Port yönlendirmeyle bir Pod'a erişim sürmesi, Service'in o Pod'a normal trafik yönlendirdiğini göstermez. Tek başına tarayıcı davranışını kanıt olarak kullanmayın.

```
kubectl scale deployment/db -n hafta6 --replicas=1
kubectl rollout status deployment/db -n hafta6 --timeout=180s
kubectl wait -n hafta6 --for=condition=Ready pod -l app=web --timeout=180s
kubectl get pods -n hafta6
kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
```

Beklenen sonuç: web tekrar hazır; Service hedefi ready: true; kayıtlar korunur. Bölüm 8'deki küme içi isteği tekrar çalıştırın ve yeni bir kayıt ekleyip listeleyin. Kanıt: kesinti öncesi, kesinti sırası ve toparlanma durumları; web yeniden başlatma sayıları.

10 Canlılık başarısızlığı ve konteynerin yeniden başlatılması

10.2 Kontrollü canlılık deneyi

Bu deneyde web kopya sayısı 1, veritabanı hazır olmalıdır. /tmp/live-fail dosyası yalnız web konteynerinin yazılabilir katmanına oluşturulur. Üç başarısız kontrolden sonra kubelet konteyneri yeniden başlatır; yeni konteynerde geçici dosya bulunmaz ve uygulama toparlanır.

```
$webPod = kubectl get pods -n hafta6 -l app=web -o jsonpath='{.items[0].metadata.name}'
kubectl get pod $webPod -n hafta6 -o jsonpath='{.metadata.uid}'
kubectl get pod $webPod -n hafta6
kubectl exec -n hafta6 $webPod -- python -c "from pathlib import Path; Path('/tmp/live-fail').touch()"
kubectl get pod $webPod -n hafta6 -w
```

RESTARTS arttığında ve web yeniden 1/1 olduğunda Ctrl+C ile yalnız izlemeyi durdurun. Deney genellikle birkaç kontrol aralığında tamamlanır; yeniden başlatma gecikmesi ortamınıza bağlıdır. Başlangıç ve hazır olma kontrolleri tekrar tamamlanmalıdır.

```
kubectl get pod $webPod -n hafta6 -o jsonpath='{.metadata.uid}'
kubectl describe pod $webPod -n hafta6
kubectl logs $webPod -n hafta6 --previous --tail=20
kubectl exec -n hafta6 $webPod -- python -c "from pathlib import Path; print(Path('/tmp/live-fail').exists())"
```

Beklenen sonuç: Pod adı ve UID aynı; RESTARTS artmış; describe olaylarında canlılık başarısızlığı ve konteynerin yeniden başlatılması görülür. Son komut False döndürür. Pod yeni UID aldıysa bu deney sırasında başka bir Pod yenileme işlemi olmuştur; koşulları kontrol edip tekrarlayın. Aynı Pod içindeki konteyner yeniden başlatması ile Bölüm 9'daki yeni Pod oluşturulmasını karşılaştırın.

Canlılık deneyindeki işaret dosyası üretim sağlık tasarımının parçası değildir; güvenli biçimde farklı Kubernetes tepkilerini gözlemlemek için kullanılır. Konteyner toparlanmazsa durum ve olayları inceleyin. Çalışan konteynerde dosya kalmışsa yalnız bu işaret dosyasını unlink ile kaldırın; PVC veya gizli dosyaları silmeyin. Port yönlendirmeyi yeniden açıp kayıtları doğrulayın.

11 Kaynakları kontrol etme ve elle ölçekleme

```
kubectl describe deployment web -n hafta6
kubectl scale deployment/web -n hafta6 --replicas=2
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get deployment web -n hafta6
kubectl get pods -n hafta6 -l app=web -o wide
kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
```

Beklenen sonuç: web READY=2/2; iki web Pod'u ve iki hazır Service hedefi. db tek kopya kalır. describe çıktısındaki CPU ve bellek requests (talepler) ve limits (sınırlar) değerlerini dosyayla karşılaştırın. İkinci Pod Pending ise describe pod ve olaylardan yetersiz kaynak, yerleştirme veya imaj erişim nedenini belirleyin [9].

Bölüm 8'deki check isteğini birkaç kez çalıştırın; her isteğin başarılı olduğunu kontrol edin. Tarayıcıdan bir kayıt daha ekleyip listeleyin. Bu sağlık yanıtı hangi Pod'un cevap verdiğini bildirmez; birkaç başarılı istek trafiğin eşit dağıtıldığını kanıtlamaz. İki hazır hedef ve Service üzerinden başarılı erişimi birlikte gösterin.

```
kubectl get pods -n hafta6 -l app=db
kubectl scale deployment/web -n hafta6 --replicas=1
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get deployments,pods -n hafta6
kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
```

Beklenen sonuç: web READY=1/1; fazla web Pod'u kaldırılmış; db hâlâ 1 kopya. Sonlanan Pod port yönlendirmesinin hedefiyse bağlantı biter; oturumu yeniden açın ve erişimi kontrol edin. Kanıt: 1, 2 ve tekrar 1 hazır web kopyası; Service hedefleri ve erişim.

Bu çalışmada kopya sayısı elle değiştirilir; otomatik ölçekleme kurulumu zorunlu değildir. Web kopyalarını artırmak veritabanını çoğaltmaz. Aynı veritabanı dosyalarına bağımsız PostgreSQL kopyaları bağlayarak ölçekleme yapmayın.

Komutla yapılan ölçekleme dosyanın replicas alanını değiştirmez. 05-web.yaml yeniden uygulanırsa replicas: 1 geçerli olur. Deney sonundaki istenen durum zaten 1 olduğu için dosyayla tekrar uyumludur.

12 Yeni sürüme geçiş

12.1 Yeni imajı hazırlama

```
notepad .\app.py
```

APP_VERSION değerini "1.2" yapıp kaydedin. Tarayıcıda hâlâ 1.1 görülmelidir; kaynak dosyasının değiştirilmesi çalışan imajı değiştirmez. Veritabanı şemasına dokunmayın.

```
docker build -t dersnotlari-web:1.2 .
docker image ls dersnotlari-web
```

Minikube kullanıyorsanız minikube image load -p hafta6 dersnotlari-web:1.2 çalıştırın. kind kullanıyorsanız Bölüm 3'te doğruladığınız küme adıyla kind load docker-image dersnotlari-web:1.2 --name desktop çalıştırın. kubeadm ana yolunda yerel imaj görünürlüğü geçerlidir; indirme hatasıyla devam etmeyin.

12.2 Kademeli güncellemeyi izleme

Ayrı bir terminalde kubectl get pods -n hafta6 -l app=web -w ile izlemi başlatın. Asıl terminalde:

```
kubectl rollout history deployment/web -n hafta6
kubectl set image deployment/web web=dersnotlari-web:1.2 -n hafta6
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get pods -n hafta6 -l app=web
kubectl get deployment web -n hafta6 -o jsonpath='{.spec.template.spec.containers[0].image}'
kubectl rollout history deployment/web -n hafta6
```

Beklenen sonuç: yeni Pod önce hazır olur, sonra eski Pod kaldırılır; çalışan imaj 1.2 olur. İzlemeyi Ctrl+C ile kapatın. Port yönlendirmeyi yeniden açın; ekranda ve /health/ready yanıtında 1.2 görülmelidir. Önceki kayıtlar korunur. Güncelleme sonrası 1.2 kaydı başlığıyla ek bir kayıt oluşturun; geri dönüşte bunu da kontrol edeceksiniz.

Kanıt: rollout durumu, imaj etiketi, sürüm ekranı ve mevcut kayıtlar. Rollout'un tamamlanması bütün uygulama işlevlerinin doğru olduğunu kanıtlamaz; kayıt ekleme ve listeleme ayrıca yapılır. maxUnavailable: 0 kesintisizlik garantisi değildir; kapasite, sağlık ve bağlantı koşulları sonucu etkiler [11].

13 Önceki sürüme dönüş ve doğrulama

Bu aşamada başka bir Pod şablonu değişikliği yapmadığınız için bir önceki revizyon 1.1'dir. Önce geçmişi kontrol edin; farklı değişiklik yaptıysanız history içindeki ilgili revizyonu inceleyip --to-revision ile doğru 1.1 revizyonunu seçin.

```
kubectl rollout history deployment/web -n hafta6
kubectl rollout undo deployment/web -n hafta6
kubectl rollout status deployment/web -n hafta6 --timeout=180s
kubectl get deployment web -n hafta6 -o jsonpath='{.spec.template.spec.containers[0].image}'
kubectl get pods -n hafta6 -l app=web
```

Beklenen sonuç: web yeniden dersnotlari-web:1.1 imajını kullanır. Port yönlendirmeyi yeniden açın; tarayıcıda ve sağlık yanıtında 1.1 görünür. Kalıcılık A, Kalıcılık B ve 1.2 sürümünde eklediğiniz yeni kayıt dâhil önceki kayıtların hepsi bulunmalıdır.

Uygulama imajına dönüş, veritabanı kayıtlarını geri almaz. Aynı adla kullanılan ConfigMap veya Secret içeriklerini de kendiliğinden eski hâline getirmez. Önceki imajın erişilebilir olması ve mevcut yapılandırma/veritabanıyla uyumlu çalışması gerekir [11].

```
kubectl exec -n hafta6 check -- python -c "import urllib.request; print(urllib.request.urlopen('http://web:8000/health/ready', timeout=10).read().decode())"
```

Beklenen sonuç: küme içi Service isteği ready ve 1.1 yanıtını alır. Kanıt: geri dönüş sonucu, çalışan imaj, sürüm ekranı ve 1.2 sırasında oluşturulan kaydın korunması. Kubernetes dosyasında 1.1 zaten temel sürümdür; kaynak app.py ise derleme deneyi sonrasında 1.2 olabilir. Çalışan imajla yerel kaynak dosyasını karıştırmayın.

13.1 Güncelleme ilerlemiyorsa

```
kubectl describe deployment web -n hafta6
kubectl get pods -n hafta6 -l app=web
kubectl get events -n hafta6 --sort-by=.metadata.creationTimestamp
```

Yeni imajın düğümlerde bulunmasını, Pod'un kaynak kapasitesini ve hazır olma kontrolünü inceleyin. İlerleme zaman aşımı otomatik geri dönüş anlamına gelmez. Sorunu düzelterek tekrar doğrulayın veya doğru eski revizyona bilinçli olarak dönün. PVC'yi silmek güncelleme sorunlarının genel çözümü değildir.

14 Sık karşılaşılan sorunlar

Belirti	İlk kontrol ve düzeltme
ErrImageNeverPull	İmaj aynı kümenin düğümlerinde mi? Bölüm 3 yolunu ve yeni etiketin yüklenmesini kontrol edin.
ImagePullBackOff db	postgres:16 için ağ erişimi ve indirme olaylarını inceleyin.
PVC Pending	StorageClass adı, varsayılan sınıf ve tüketici bekleme davranışını; Pod olaylarını kontrol edin.
Web Running ama 0/1	db hazır mı; DB_HOST=db ve port 5432 mi? Secret anahtarları ve dosya yollarını kontrol edin.
Parola hatası	İlk depolama hazırlığında kullanılan parola dosyasını koruyun. Secret değişikliği mevcut veritabanı parolasını değiştirmez.
CrashLoopBackOff	kubectl logs --previous ve describe pod olaylarıyla nedenini inceleyin; eksik APP_SECRET veya dosya erişimini kontrol edin.
Tarayıcı bağlantısı bitti	Port yönlendirme oturumu ve hedef Pod yenilenmiş olabilir; Bölüm 7 komutunu yeniden başlatın.
İkinci web Pod Pending	İmaj yükleme ve kullanılabilir düğüm kapasitesi; requests değerleri ve yerleştirme olayları.
Kayıtlar kayıp	PVC/PV aynı mı; namespace ve PGDATA doğru mu? PVC silinmişse yeni alan oluşmuş olabilir.
Form doğrulanmıyor	Sayfayı yenileyin; bütün web kopyaları aynı APP_SECRET kullanmalı. Yerel HTTP için COOKIE_SECURE=false.
check exec çalışmıyor	Kontrol Pod'u Running mı? Bir günlük bekleme süresi bitmişse yalnız check Pod'unu yeniden oluşturun.

```
kubectl get pods -n hafta6
kubectl get events -n hafta6 --sort-by=.metadata.creationTimestamp
kubectl logs -n hafta6 deployment/web --tail=30
kubectl describe pod -n hafta6 <Pod-adi>
```

Son komuttaki <Pod-adi> yerine gerçekten görünen adı yazın; köşeli yer tutucuyu çalıştırmayın. Durum, olay ve günlükler farklı bilgiler verir. Rapora alırken gizli bilgi ve kişisel verilerin görünmediğini kontrol edin.

15 Çalışmayı sonlandırma ve depolamayı koruma

15.1 Kanıtları ve dosyaları saklama

Önce rapor kanıtlarını alın. Port yönlendirme ve izleme pencerelerini Ctrl+C ile kapatın. app.py, Dockerfile, .dockerignore, k8s ve secrets dosyalarını bilgisayarınızda koruyun; secrets klasörünü paylaşılabilir teslimin dışında tutun.

15.2 Uygulama kaynaklarını kaldırıp PVC koruma

```
kubectl delete deployment web db -n hafta6
kubectl delete service web db -n hafta6
kubectl delete pod check -n hafta6
kubectl delete configmap app-config -n hafta6
kubectl delete secret app-secrets -n hafta6
kubectl get deployments,pods,services,configmaps,secrets -n hafta6
kubectl get pvc db-data -n hafta6 -o wide
kubectl get pv
```

Beklenen sonuç: uygulamaya ait Deployment, Pod, Service, ConfigMap ve Secret kaldırılır; db-data PVC ve bağlı PV korunur. Ad alanındaki otomatik kube-root-ca.crt ConfigMap'i gibi sistem nesneleri kalabilir; bunlar uygulama tanımları değildir. Namespace'i bu aşamada silmeyin; silinmesi PVC'yi de kaldırır.

15.3 Korunan verilerle yeniden çalışma

Koruma sonucunu ayrıca göstermek için Bölüm 4.1 ve 4.2 ile ConfigMap/Secret'i aynı yerel dosyalardan yeniden oluşturun. Bölüm 7.1 ile db, Service ve web dosyalarını yeniden uygulayın. 02-pvc.yaml yeniden uygulanabilir; var olan db-data talebini kullanır. Aynı parola ve aynı PVC ile eski kayıtlar görünmelidir. Tarayıcı kanıtını aldıktan sonra Bölüm 15.2'yi tekrar uygulayın.

Bu yeniden açma, yalnız uygulama kaynaklarının silinmesinin veritabanını sildiği anlamına gelmediğini gösterir. Yerel imajlar ve çalışma dosyaları Kubernetes nesnelerinden farklı bir yaşam döngüsüne sahiptir.

16 Kalıcı veriyi silme seçeneği

Bu bölümdeki silme işlemi, yalnız kayıtların artık gerekli olmadığına karar verdiğinizde uygulanır. Veriyi korumayı seçmeniz de ders notunun başarı ölçütünü karşılar; raporda seçiminizi ve sonucunu belirtin. PVC silme, Pod silme deneyinin yerine geçmez.

16.1 Silmeden önce politika kontrolü

```
$pvName = kubectl get pvc db-data -n hafta6 -o jsonpath='{.spec.volumeName}'
kubectl get pv $pvName
kubectl get pv $pvName -o jsonpath='{.spec.persistentVolumeReclaimPolicy}'
```

Delete politikası, PVC silinince PV ve desteklenen altyapı depolamasının kaldırılmasına yol açabilir. Retain (koru) politikasında PV/veri korunur ve ayrıca yönetilmelidir. Verilerin gerektiği durumda yedek almadan bu adıma geçmeyin [6].

16.2 Yalnız bu çalışmanın PVC nesnesini silme

Bölüm 15.2 ile tüketici Pod'ları ve Deployment'ları kaldırılmış olmalıdır. Veriyi silme seçimini yaptıysanız:

```
kubectl delete pvc db-data -n hafta6
kubectl get pvc -n hafta6
kubectl get pv $pvName
```

Beklenen sonuç: db-data artık listede yoktur. Delete politikasında PV kaldırılabilir ve son komut NotFound verebilir. Retain politikasında PV Released kalabilir; bu verinin de silindiğini göstermez. Yerel depolama sürücüsünün durumunu inceleyin; başka çalışmalara ait PV veya dizinleri silmeyin. Namespace ancak içindeki gerekli kaynaklar tamamen değerlendirildikten sonra ayrıca kaldırılabilir.

Yeni PVC oluşturarak tekrar dağıtım yapmak eski veriyi otomatik geri getirmez. Yeni bir veri alanında önceki kayıtlar bulunmaz. Eski kayıtları geri yüklemek ayrı bir yedekleme/veri aktarımı işlemidir. Hafta 5 Docker volume'undaki veriler de bu işlemlerden bağımsızdır.

17 Teslim ve görevlerin eksiksizliği

Kısa raporda her kanıtın altına yaptığınız işlemi, beklediğiniz sonucu ve gerçekten gözlediğiniz sonucu yazın. Hata olduysa belirtiyi, incelediğiniz durumu, yaptığınız düzeltmeyi ve yeniden doğrulama sonucunu ekleyin. Hazır kaynak kodları ve YAML dosyalarının ayrıca teslimi zorunlu değildir; gerektiğinde istenebilir.

Ders notundaki görev	Kılavuz adımı ve teslim kanıtı
7.1 Dağıtım ve bağlantı	2–8: imaj; Deployment/Service; ConfigMap/Secret; PVC/PV; hazır Pod'lar ve kayıt ekranı.
7.1 İşlevler ve kalıcılık	7 ve 9.1: kayıt ekleme/listeme; farklı db Pod UID; aynı PVC ve aynı kayıtlar.
7.1 Sağlık ve yeniden oluşturma	9.2 ve 10: yeni web Pod'u; readiness/EndpointSlice değişimi; aynı Pod UID ile artan RESTARTS.
7.1 Kaynak ve ölçekleme	6.3 ve 11: requests/limits; 1 → 2 → 1 hazır kopya; Service erişimi; db tek kopya.
7.1 Güncelleme ve geri dönüş	12–13: 1.1 → 1.2 → 1.1; imaj/sürüm kanıtı; güncellemeden sonra eklenen kaydın korunması.
7.1 Kaynakları sonlandırma	15–16: uygulama nesnelerinin kaldırılması; PVC koruma veya silme seçimi ve politika sonucu.
7.3 Kavramsal açıklamalar	Aşağıdaki altı soru; kendi nesneleriniz ve deney kanıtlarınız üzerinden yanıt.
7.3 Dosyalar ve gizlilik	Çalışma dosyalarını koruma; raporda parola/anahtar bulunmaması. Bulut kanıtları Ek 3 kapsamında.

Kavramsal açıklama soruları

1 Web uygulamasına erişim ile web bileşeninin PostgreSQL'e erişimi nasıl sağlanır?

2 Silinen web Pod'unun yerine neden yeni Pod oluşturulur? Konteyner yeniden başlatılması ile Pod yeniden oluşturulması arasındaki fark nedir?

3 Kayıtlar nerede saklanır? PostgreSQL Pod'u yeniden oluşturulduğunda neden korunur?

4 Hazır olma kontrolünün başarısızlığı ile canlılık kontrolünün başarısızlığı uygulamayı nasıl etkiler?

5 Web kopya sayısı değiştiğinde hangi bileşenler değişir? PostgreSQL neden aynı şekilde çoğaltılmaz?

6 Yeni sürüm ve geri dönüş nasıl doğrulanır? Önceki uygulama sürümüne dönmek neden veritabanını da önceki hâline getirmez?

18 Kaynaklar

Kaynak numaraları bu bağımsız ek içinde 1'den başlar. Komutların ilgili resmî başvuruları aşağıdadır. Yerel uygulama değerleri Hafta 5 kaynak paketiyle eşleştirilmiştir.

[1] Docker Desktop Kubernetes
https://docs.docker.com/desktop/use-desktop/kubernetes/

[2] Minikube image komutları
https://minikube.sigs.k8s.io/docs/commands/image/

[3] kind imaj yükleme
https://kind.sigs.k8s.io/docs/user/quick-start/

[4] Kubernetes ConfigMap
https://kubernetes.io/docs/concepts/configuration/configmap/

[5] Kubernetes Secret
https://kubernetes.io/docs/concepts/configuration/secret/

[6] Kubernetes kalıcı birimler
https://kubernetes.io/docs/concepts/storage/persistent-volumes/

[7] Kubernetes Service ve hedefler
https://kubernetes.io/docs/concepts/services-networking/service/

[8] Kubernetes sağlık kontrolleri
https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[9] Kubernetes kaynak yönetimi
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[10] Kubernetes port yönlendirme
https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/

[11] Kubernetes Deployment
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/